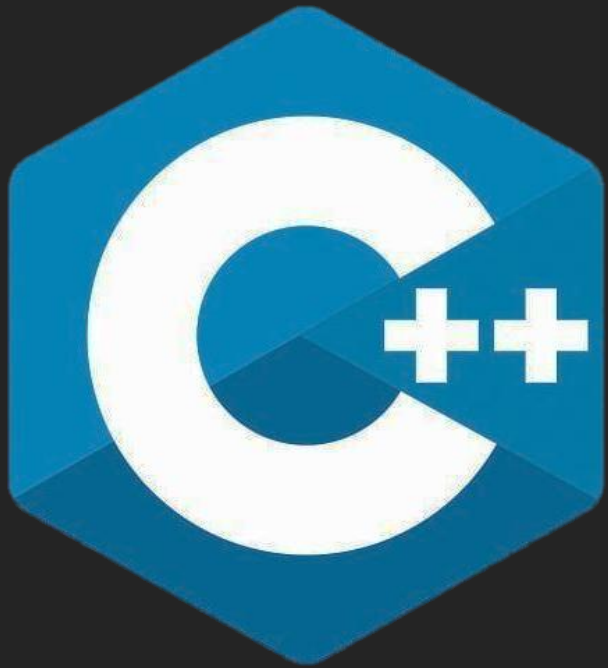




Cs50
Data
Structure

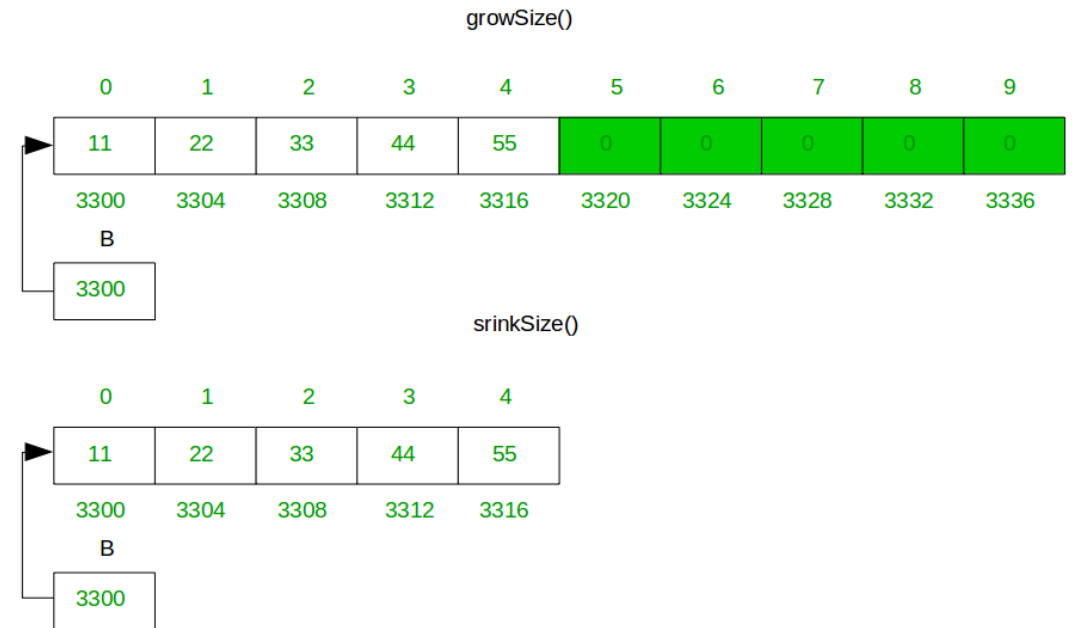


- Vector
- Vector 2d
- Stack
- Queue



Vector

- Dynamic array that resizes automatically
- Stores elements contiguously for fast access
- Amortized $O(1)$ time complexity for `push_back`, and `pop_back`
- Constant time random access using `[]` or `at()`



Vector Functions

1. `push_back(value)`: Add element at the end
2. `pop_back()`: Remove last element (no return)
3. `size()`: Get number of elements
4. `empty()`: Check if vector is empty
5. `at(index)`: Access element with bounds checking
6. `operator[](index)`: Access element without bounds check



Vector Functions



```
1  #include<iostream>
2  #include<vector>
3
4  using namespace std;
5
6  int main(void){
7
8      vector<int> nums = {1, 2, 3, 4};
9
10     nums.push_back(5);
11
12     for(int i = 0; i < nums.size(); i++){
13         cout << nums[i] << " ";
14     }
15     cout << "\n";
16
17     return 0;
18 }
```

You must include "vector" library

Note how to define the data type of the vector

You can use the "push_back" function in this way to add an element to the end of the vector

You can use the size function to get the current size of the vector.
We used that function to know how many elements we need to iterate

You can access a specific element using operator "["
Yes, it is as same as the Array.



```
1  output :
2  1 2 3 4 5
```



Vector Functions

```
1  #include<iostream>
2  #include<vector>
3
4  using namespace std;
5
6  int main(void){
7
8      vector<int> nums = {1, 2, 3, 4, 5, 6};
9
10     nums.pop_back();
11
12     for(int i = 0; i < nums.size(); i++){
13         cout << nums.at(i) << " ";
14     }
15
16     return 0;
17 }
```

You can use the `pop_back` function in this way to remove an element from the end of the vector

You can use the `size` function to get the current size of the vector.
We used that function to know how many elements we need to iterate

You can access a specific element using function `at`

Note is give the same output like the previous code because we have deleted the last element

```
1  output :
2  1 2 3 4 5
```



Vector Functions



```
1  #include<iostream>
2  #include<vector>
3
4  using namespace std;
5
6  int main(void){
7
8      vector<int> nums(3);
9      nums[0] = 1;
10     nums[1] = nums[0] + 5;
11     cin >> nums[2];
12
13     for(int i = 0; i < nums.size(); i++){
14         cout << nums.at(i) << " ";
15     }
16
17     return 0;
18 }
```

This example shows some functionality and operations you can apply on vectors

Note how we Initialize the vector with a specific size in this example



Vector

Task:

Remove even numbers from a given vector {1, 2, 3, 4, 5, 6}



Vector

Task:

Remove even numbers from a given vector {1, 2, 3, 4, 5, 6}

Solution:

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      vector<int> nums = {1, 2, 3, 4, 5, 6};
7
8      vector<int> oddOnly;
9      for (int num : nums)
10         if (num % 2 != 0)
11             oddOnly.push_back(num);
12
13     cout << "Odd numbers: ";
14     for (int num : oddOnly)
15         cout << num << " ";
16
17     return 0;
18 }
```



2d Vector

Represent matrices or grids efficiently with nested vectors.

Each element itself is a vector, supporting flexible row sizes.

Example:

```
1  
2  
3  vector<vector<int>> matrix;  
4  
5  
6  matrix.push_back({1, 2, 3});  
7  
8  
9  matrix[row][col];
```

Add row:

Access element:

	Col 1	Col 2	Col 3	Col 4
Row 1	x[0][0]	x[0][1]	x[0][2]	x[0][3]
Row 2	x[1][0]	x[1][1]	x[1][2]	x[1][3]
Row 3	x[2][0]	x[2][1]	x[2][2]	x[2][3]



2d Vector



```
1  #include<iostream>
2  #include<vector>
3
4  using namespace std;
5
6  int main(void){
7
8      vector<vector<int>> nums(3, vector<int>(3));
9      nums[0] = {1, 2, 3};
10     nums[1] = {4, 5, 6};
11     nums[2] = {7, 8, 9};
12
13     cout << nums[1][1] << "\n";
14
15     nums[1][1] = 100;
16
17     cout << nums[1][1] << "\n";
18
19     return 0;
20 }
```

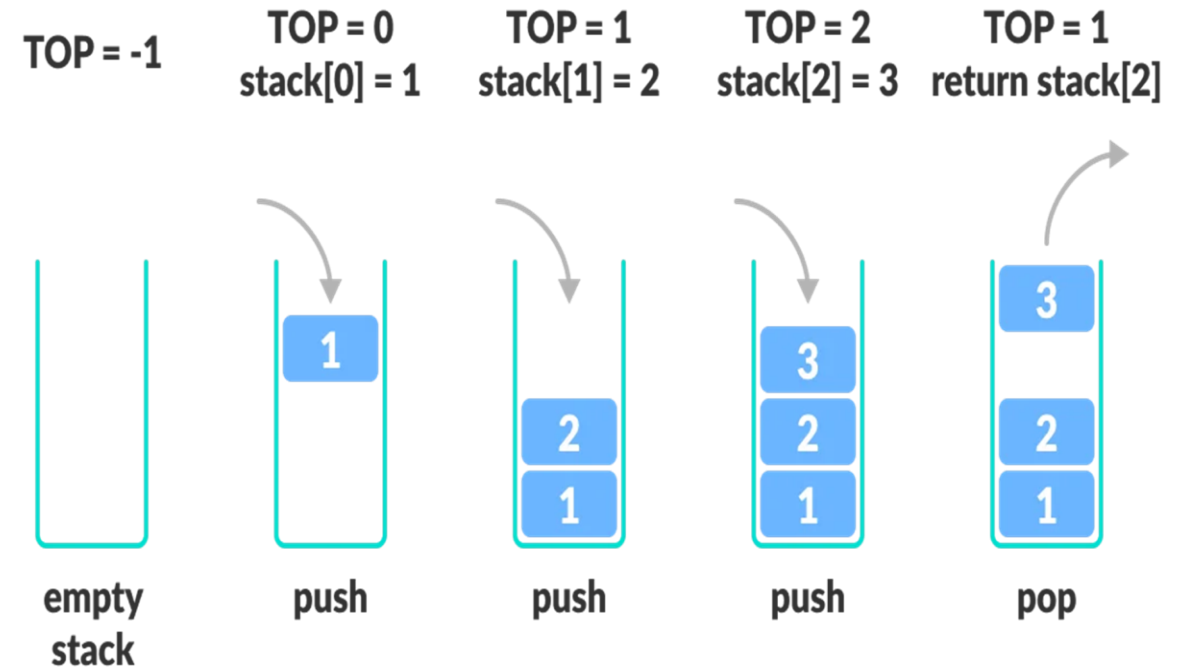
Let's try to run this code and note what will happen when we change some values



Stack

A **stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle, where the last element added is the first to be removed.

It supports two main operations: **push** (to add an element) and **pop** (to remove the top element).



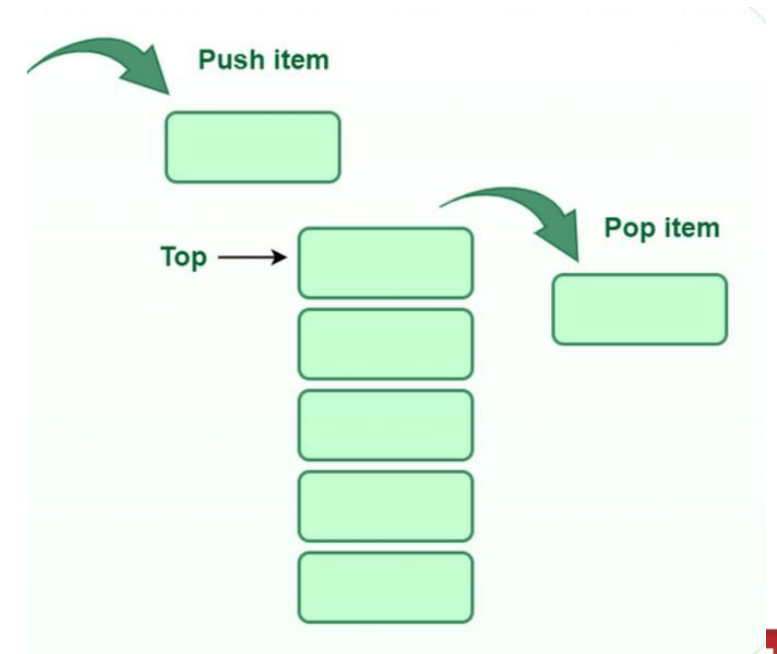
Stack

- the important rule is that when data is added to the stack, it sits “on top,” and so if an element needs to be removed, the most recently added element is the only element that can legally be removed.

Last in, first out (LIFO)

There are operations that may legally be performed on a stack.

- push: Add a new element to the top of the stack.
- pop: Remove the most recently-added element from the top of the stack.
- top() Returns the top element of the stack.
- empty() returns true if stack is empty else false.
- size() returns the size of stack.



Stack

You must include “stack” library

Note how to define the data type of the stack

You can use the “push” function in this way to add an element to the top of the stack

Use the “top” function to access the top element of the stack

Use “pop” function to remove the top element from the stack

You know that the stack has 3 elements so you can repeat that code 3 times, but what will happen if we repeat that code more than 3 times?

```
1  #include<iostream>
2  #include<stack>
3
4  using namespace std;
5
6  int main(void){
7
8      stack<int> st;
9      st.push(1);
10     st.push(2);
11     st.push(3);
12
13     cout << st.top() << "\n";
14     st.pop();
15
16     cout << st.top() << "\n";
17     st.pop();
18
19     cout << st.top() << "\n";
20     st.pop();
21
22     return 0;
23 }
```



Stack

We can write the same functionality in different code

```
1  #include<iostream>
2  #include<stack>
3
4  using namespace std;
5
6  int main(void){
7
8      stack<int> st;
9      st.push(1);
10     st.push(2);
11     st.push(3);
12
13     while(!st.empty()){
14         cout << st.top() << "\n";
15         st.pop();
16     }
17
18     return 0;
19 }
```



Stack

Task:

Reverse a string using a stack



Stack

Task:

Reverse a string using a stack

Solution:

```
1  #include <iostream>
2  #include <stack>
3  using namespace std;
4
5  int main() {
6      string str = "hello";
7      stack<char> s;
8
9      for (char ch : str)
10         s.push(ch);
11
12     cout << "Reversed string: ";
13     while (!s.empty()) {
14         cout << s.top();
15         s.pop();
16     }
17 }
```



Queue

A **queue** is a linear data structure that follows the **First In, First Out (FIFO)** principle, where the first element added is the first to be removed.

It supports two main operations: **enqueue** (to add an element at the rear) and **dequeue** (to remove the element from the front).



Queue

The important rule is that when data is added to the queue, it is tacked onto the back, and so if an element needs to be removed, the element at the front is the only element that can legally be removed.

First in, First out (FIFO)



Queue

- **There are operations that may legally be performed on a queue.**

push(): Add a new element to the end of the queue.

pop(): Remove the oldest element from the front of the queue.

front(): Access front element without removing

back(): Access back element without removing

empty() : Checks if the queue is empty

size(): This operation returns the size of the queue i.e. the total number of elements it contains.

When multiple documents are sent to a printer, they are added to a queue. The printer processes them in the order they arrive — the **first document sent** is the **first to be printed**, just like **FIFO (First In, First Out)** behavior.



Queue

You must include “queue” library

Note how to define the data type of the queue

You can use the “push” function in this way to add an element to the end of the queue

Use the “front” function to access the first/front element of the queue

Use “pop” function to remove the first/front element from the stack

You know that the queue has 3 elements so you can repeat that code 3 times, but what will happen if we repeat that code more than 3 times?

```
1  #include<iostream>
2  #include<queue>
3
4  using namespace std;
5
6  int main(void){
7
8      queue<int> q;
9
10     {
11         q.push(1);
12         q.push(2);
13         q.push(3);
14     }
15     cout << q.front() << "\n";
16     q.pop();
17     cout << q.front() << "\n";
18     q.pop();
19     cout << q.front() << "\n";
20     q.pop();
21
22     return 0;
23 }
24 }
```

Queue

We can write the same functionality in different code

```
1  #include<iostream>
2  #include<queue>
3
4  using namespace std;
5
6  int main(void){
7
8      queue<int> q;
9
10     q.push(1);
11     q.push(2);
12     q.push(3);
13
14     while(!q.empty()){
15         cout << q.front() << "\n";
16         q.pop();
17     }
18
19     return 0;
20 }
```



Queue

Task:

Simulate a simple line of people



Queue

Task:

Simulate a simple line of people

Solution:

```
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4
5  int main() {
6      queue<string> line;
7      line.push("Alice");
8      line.push("Bob");
9      line.push("Charlie");
10
11     cout << "Serving order:\n";
12     while (!line.empty()) {
13         cout << line.front() << endl;
14         line.pop();
15     }
16 }
17
```





Support
Team

